

Evolving research question: can ML algos determine the mathematical equations behind a physical system just by watching it move even when the data is messy/noisy? Do some algos handle that better than others?

3 systems being tested:

- Lorenz system: “Butterfly” shape, small differences -> completely different long term results. 3-D
- Van der Pol oscillator: self regulating/repeating loop that always settles into a rhythm: if motion is too small, absorbs energy; if motion too large, releases.
- Duffing oscillator: pushed by outside force on regular rhythm, reacts in nonlinear way.

systems.py - Code that simulates above 3 systems using their real equations, and can later add random noise/delete data points

validate_systems.py - draws picture of the systems for human validation

SINDy: Sparse Identification of Nonlinear Dynamics. Feed a large amount of mathematical blocks, use stats to figure out which blocks explain how the system moves. Tries to get the mathematical model of each system.

sindy_from_scratch.py - simplified version of SINDy written from scratch, ran on Lorenz data and rediscovered Lorenz equations correctly. (before noisy data)

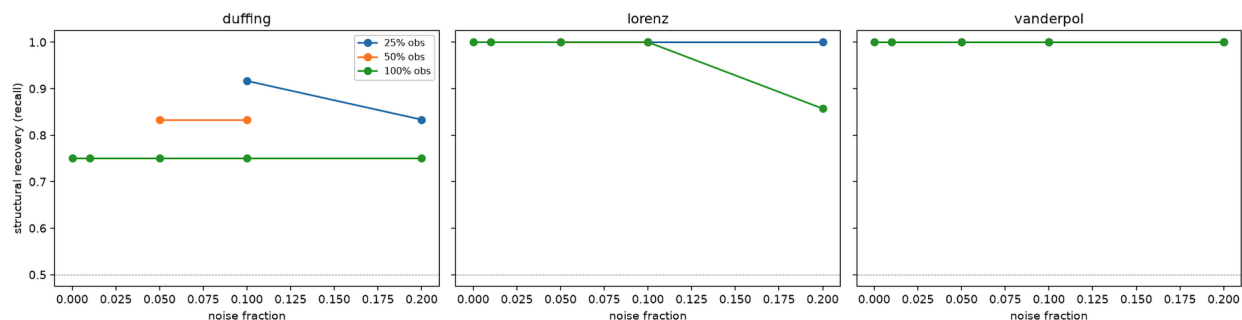
scoring.py - grades SINDy on whether it found the actual math equation.

check_pysindy.py - to check whether the real PySINDy library agrees with the handwritten version.

sindy_grid.py - runs SINDy with every noise/corruption combination for all 3 systems, 3 times each with different random corruption, and saves to a file.

plot_failure_boundary.py - turns file from sindy_grid.py to a plot that is readable.

First run plot -

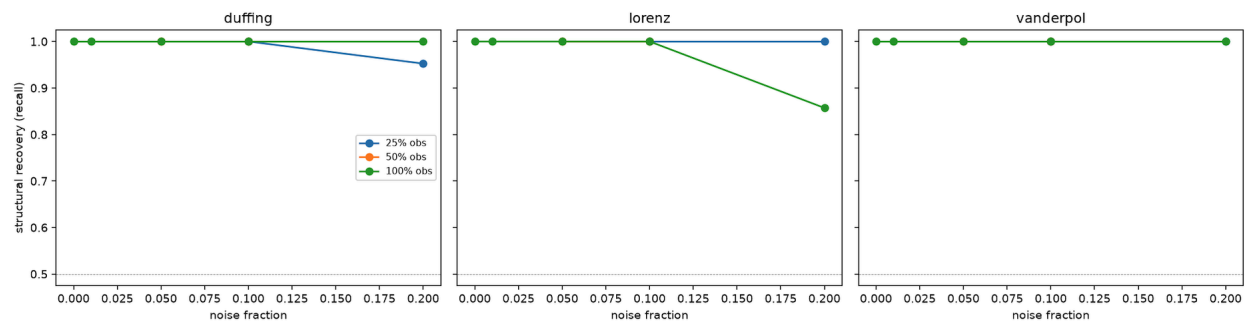


Observations:

- Van der Pol never drops below 1.0 even at max noise. -> grid not harsh enough
- Orange line missing for lorenz data
- Duffing is always at ~0.75 no matter the noise level (maybe because Duffing depends on time?)

Duffing forcing term broke recovery even at 0% noise -> fixed through state augmentation
Discovered a collinearity bug in the augmented state -> fixed through split library.

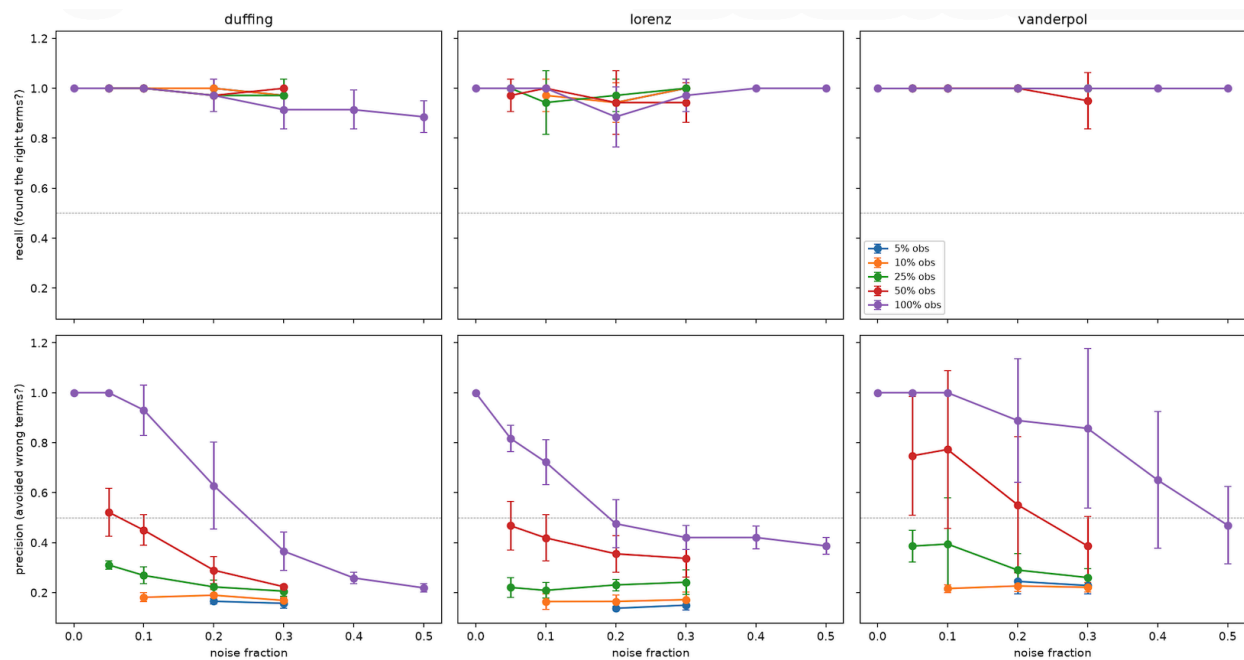
Second run -



Almost everything sits at 1.0 recall, only lorenz has real degradation (possibly due to chaos?)

Does chaos, not just noise, predict how fragile equation discovery is?

3rd -



Recall barely moves but precision collapses

- Collapses more from low obs than noise

Sparsity is dominant driver of SINDy's failure mode in this case, not noise

- Failure mode: invents extra terms

Gap in evidence - recall near 1.0 while precision drops is consistent with both SINDy finding right terms plus noise, and model going non sparse and keeping candidate terms

- In latter case, recall = 1.0 is trivial side effect

Sparsity bloat check(5% obs/30% noise): recovered model kept 47-60 of 60 possible terms, coefficients in the hundreds, `coeff_error` > 5. Confirmed recall = 1 is trivial and not a genuine recovery. The model has abandoned sparsity.

Hypothesis: column normalization = cause. Tested `normalize_columns` = True on the same condition, made it worse (60/60 terms kept). Don't think scaling is the explanation. 100 noisy points can't support a reliable derivative estimate at all.

Cliff search: single-seed sweep across `obs_frac` was too noisy to read. Reran averaged over 5 seeds -> found no sharp cliff. Precision degrades smoothly as `obs_frac` drops, flattening at a floor around 0.15 below 15–20% observations; recall stays robust (>0.83) throughout.

Potential sentence in the results section of paper? -> SINDy's structural precision degrades gradually and predictably with data sparsity, plateauing near a low but nonzero floor, while recall remains largely robust throughout

PySINDy's ensembling API changed from a `.fit()` flag to a dedicated `EnsembleOptimizer` wrapper class between versions

SINDy summary (Lorenz, representative of the chaotic case):

- Structural recall stays robust (>0.83) across nearly the entire grid tested.
- Precision degrades smoothly as observations drop, flattening at a floor of ~0.14–0.16 below ~15–20% observations. Declines to a low plateau.
- At the most extreme sparsity, the model fails by abandoning sparsity entirely (up to 60/60 candidate terms kept, coefficients off by 500%+). Confirmed through direct inspection
- Ruled out: column-scaling issues (`normalize_columns` made it worse).
- Ruled out: off-the-shelf Ensemble-SINDy (bagging) does not meaningfully improve precision at any sparsity level tested (matches plain STLSQ within noise.)
- Working hypothesis: the bottleneck is the pointwise derivative estimate itself, which becomes unreliable under sparsity in a way that resampling can't fix. Candidate real fix: methods that avoid pointwise differentiation entirely (e.g. weak-form SINDy), or a threshold that adapts to estimated derivative uncertainty rather than being fixed.

Neural ODE

Trains neural network to find dx/dt , then integrates it forward to predict trajectory.

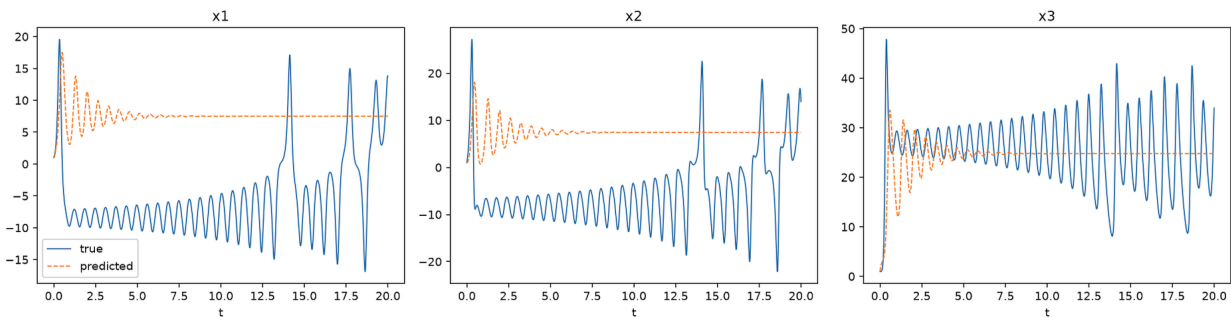
Care about prediction accuracy and long term stability.

neural_ode.py - the model and training/eval logic, trains on random windows of trajectory

check_neural_ode.py - runs on clean Lorenz data (check_sindy.py equivalent)

First run not depicted

Second run -



- normalization fixed destabilized training

- predicted trajectory oscillates a bit but then settles to a flat line ($x_1 \sim 7.5$, $x_2 \sim 7$, $x_3 \sim 24$).

 - close to Lorenz's actual fixed points

- Network has learned it as a stable equilibrium rather than its true nature as an unstable one.

- Longer windows to learn from = potential fix?

 - Nope, made it worse

- Since training run is very slow, scaled down the grid w fewer seeds and iterations

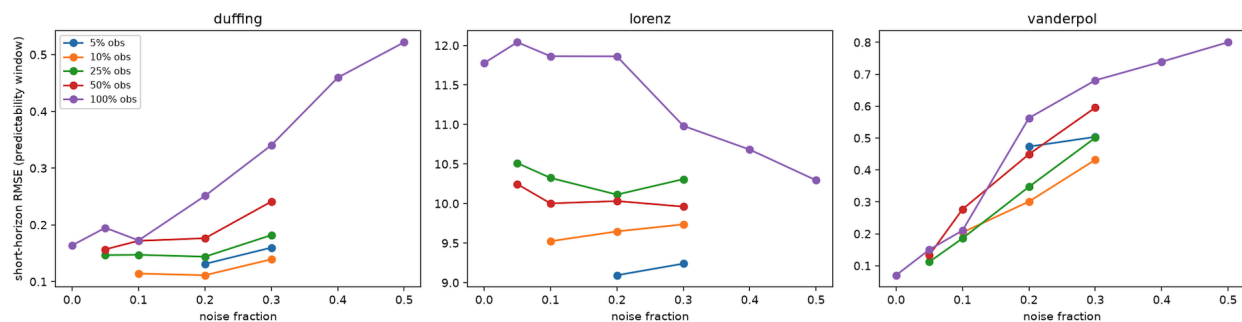
neural_ode_grid.py 1st run - Van der Pol and Duffing get small errors that barely move w noise/sparsity but Lorenz sits at 9.5-11 regardless

Finding: Neural ODE needs substantially more training compute to reach usable accuracy on a chaotic system

Neural ODE, clean-data baseline (Lorenz): after fixing a training instability caused by Lorenz's very different per-dimension scales (fixed via internal normalization), the model tracks true dynamics for roughly 1 time unit, then settles near a genuine Lorenz fixed point instead of continuing to oscillate — consistent with the known "predictability horizon" phenomenon in chaotic-system forecasting (MSE training beyond that horizon rewards hedging toward the mean over committing to an untrackable trajectory), not a bug.

Neural ODE grid (reduced: 5 conditions $\times$ 3 systems $\times$ 2 seeds, 500 iters/fit): Van der Pol and Duffing show small, stable errors largely insensitive to noise/sparsity in the range tested. Lorenz shows a flat, uniformly poor error (~ 10) across every condition — likely a training-budget ceiling (500 iters, vs. 1500 in the dedicated single-run test) rather than genuine data insensitivity. Real finding either way: Neural ODE needs substantially more compute to reach usable accuracy on

a chaotic system than on a non-chaotic one, a legitimate comparison point for the computational-cost axis of the paper.



at fixed 100% obs, Van der Pol and Duffing both show clean, strong degradation with noise — 0.07 → 0.80 for Van der Pol, 0.16 → 0.52 for Duffing (steady climbs)

- Opposite of what we found w SINDy

SINDy weakness: sparsity

Neural ODE's weakness: noise (at least in nonchaotic systems)

Neural ODE grid v1 (point-based windows) found seemingly clean noise-sensitivity result on Van der Pol/Duffing at 100% obs, but sparsity axis showed a confound: fixed point-count windows meant sparser (regularly-subsampled) data trained on longer window length times. Fixed by redefining window length in time rather than points.

PINNs:

Assumes we already know correct form of equations ('this is a lorenz') but not exact parameter values

PINN (full structure known, parameters unknown) → SINDy (structure and parameters unknown, but sparse/interpretable) → Neural ODE (nothing assumed, fully black-box)

pinn.py - the network holding the true physics parameters and training loop that combines data fitting loss w physics residual loss.

check_pinn.py - trains on clean lorenz data starting with incorrect parameters to see if it finds its way back to the true ones.

Parameters drifted away from true ones at certain points.

According to existing literature → PINNs known to struggle with chaotic systems over long time domains.

same underlying issue as Lorenz's short predictability horizon with Neural ODE

Would more iterations work?

PINN, clean Lorenz baseline: initial attempt (full 20-time-unit domain, 3000 iters) plateaued with ~35% mean parameter error — consistent with PINNs' known difficulty fitting long chaotic horizons as one smooth function ("spectral bias"). Fixed by training on a shorter window (3 time units) with more iterations (15000): converged cleanly, mean relative parameter error 0.6% (σ , ρ , β all recovered within 1.5% of true values from deliberately wrong initial guesses).

Duffing - Great

Lorenz - sparsity hurts recovery more than noise does (similar to SINDy)

Van der Pol - identical conditions give very diff answers (training not reliably converging)

Same iters for all three systems for fair comparison

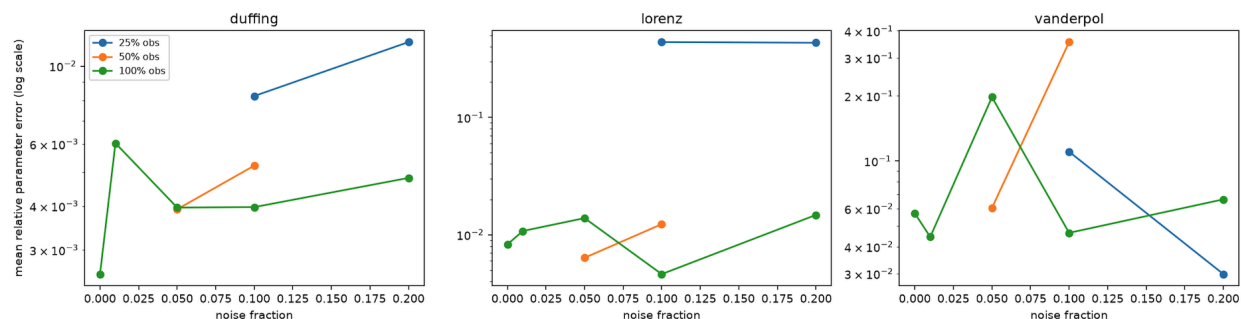
Duffing: errors under 1.5% everywhere

Lorenz: near perfect for full and 50% obs, regardless of noise up to 20%, then sharp cliff at 25% obs

Van der Pol: solid with a few outliers.

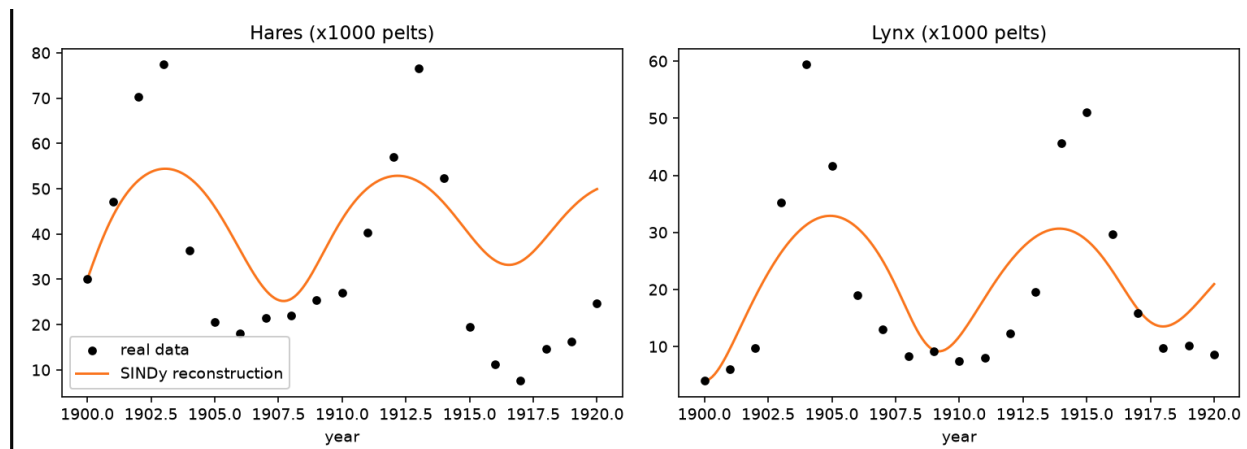
PINN is remarkably robust to noise, even up to 20%, across all three systems - Achilles heel sparsity (chaotic system)

chaos seems to amplify data-quality fragility across every method tested



On real data (Hudson Bay Company records) and a literature-reference Lotka-Volterra equation-1st run - 6 terms, with small noisy coefficients, and wrong sign term.

- Same non sparse collapse I found when i ran `check_sparsity_bloat.py` (property of SINDy?)



- Right general timing and amplitude, oscillating, but out of sync in later years.

Despite the underlying equation being structurally messy, the resulting model still qualitatively reproduces the real oscillation for roughly one and a half cycles before drifting, consistent with both the sparsity-driven identification failure and the real system's own non-stationarity.

threshold too low -> dense model with a fake bias term (0.05); threshold too high -> clean-looking but structurally wrong, missing the one term that defines the actual physics (0.1)

<https://arxiv.org/pdf/2210.08095> - documents the same result for SINDy as me, marks as "Fail", PINN does worse (also "Fail")

Improvement Phase (for TMLR submission and arXiv):

Formalized seed-level statistics for all three grids, which were previously reported as point estimates from averaged means without variance.

Regenerated Neural ODE and PINN plots with error bars, matching the treatment SINDy's plot already had.

Decision: report mean $\pm$ std everywhere going forward, per TMLR's soundness-focused review criteria, not just for SINDy.

Does the paper's conclusions survive a range of SINDy threshold values, not just 0.1?

10% noise, 25% obs: precision climbs steadily as threshold increases:

- Duffing: 0.155 -> 0.795 (threshold 0.01 $\rightarrow$ 0.50)
- Lorenz: 0.121 -> 0.448
- Van der Pol: 0.204 -> 1.000

Threshold sensitivity test -> precision floor reported for main grid is specific to threshold=0.1. Higher thresholds recover high precision at moderate sparsity (Van der Pol reaches perfect precision at threshold=0.5) but cost recall at clean conditions (Duffing recall drops to 0.714 at threshold=0.5, no noise). Benefit of higher threshold is much smaller at the harshest condition

tested.

Ranking precision by pattern: regular > random > blockEx:

- Duffing, 10% noise, 50% obs: regular = 1.000, random = 0.448, block = 0.202
- Van der Pol, 30% noise, 5% obs: regular = 0.822, random = 0.211, block = 0.215
- Lorenz, 10% noise, 25% obs: block = 0.321 beats random = 0.215 here (exception?)

Sindy fails under sparsity because of pointwise derivative estimation problem. Evenly spaced points -> predictable local neighborhood even when few points overall. Randomly scattered -> irregular local gaps that degrade estimate. Block is worst case since it is an entire stretch with zero derivative info. Confirms mechanism but specific precision floor is tied to random subsampling not sparsity.

Lorenz recall dips slightly under reg. pattern at 2 spots (only place recall isn't perf for any pattern)

We did not separately verify threshold-robustness under the regular sampling pattern.

The original hypothesis was long windows specifically push training past the predictability horizon and cause collapse. However, at modest iterations, the short window collapsed completely too (late-time std of exactly 0.00 so worse than the long window's partial 2.62). Neither collapsed at high iterations.

What window length did affect: at high iterations, the short window reached far better accuracy (RMSE 1.28 vs. 6.39) and trained about 13x faster (69s vs. 875s).

Doubling iterations (15000 -> 30000) on the two worst Van der Pol conditions:

- noise=0.05, obs=1.0: $0.076 \pm 0.038 \rightarrow 0.018 \pm 0.014$
- noise=0.10, obs=0.50: $0.043 \pm 0.026 \rightarrow 0.019 \pm 0.015$

Both mean error and variance dropped substantially. Points the same direction as the Neural ODE finding -> the instability is primarily an undertraining effect not a fundamental PINN limitation on Van der Pol.

Merged full 5-seed PINN data. Duffing and Lorenz-outside-cliff both hold under 2% error. Lorenz cliff numbers shifted with more seeds (43.8% → 26.6% mean, but std now ≈ mean). Reframed as seed-dependent failure. Separately discovered `torch.manual_seed()` was never called in either

neural method, "seed" only controls data corruption, not network init. Decided to document as a limitation rather than fix-and-rerun given remaining time.